TECHNICAL ARCHITECTURE & COMPANY EXPERT MASTER HANDBOOK

Academic Event & Resource Booking Portal

Problem Statement, Tech Stack Rationale, Alternative Analysis, Backend Security & Innovation Blueprint

Target Domain: Centralized Campus Resource & Facility Scheduling
Core Invariant: Structurally Impossible Double-Booking Guaranteed
Architecture: Layered MVC + Microservice Sidecar + Serverless Edge
Security: Dual-Token JWT + RBAC + Stateless HMAC Verification
Deployment: Vercel Edge SPA + Serverless Express + Cloud Atlas DB

Prepared for Company Technical Experts, System Architects, and Technical Interviews

Author / Lead Developer: Keerthana (Easwari Engineering College)

Version: 2.4.0 Production Release • **Date:** August 2026

1. Problem Statement

In academic institutions (engineering colleges, universities), managing high-demand shared resources—such as **Auditoriums, Seminar Halls, High-Performance Computing Labs, Smart Classrooms, and Mobile AV Kits**—is plagued by fundamental operational inefficiencies:

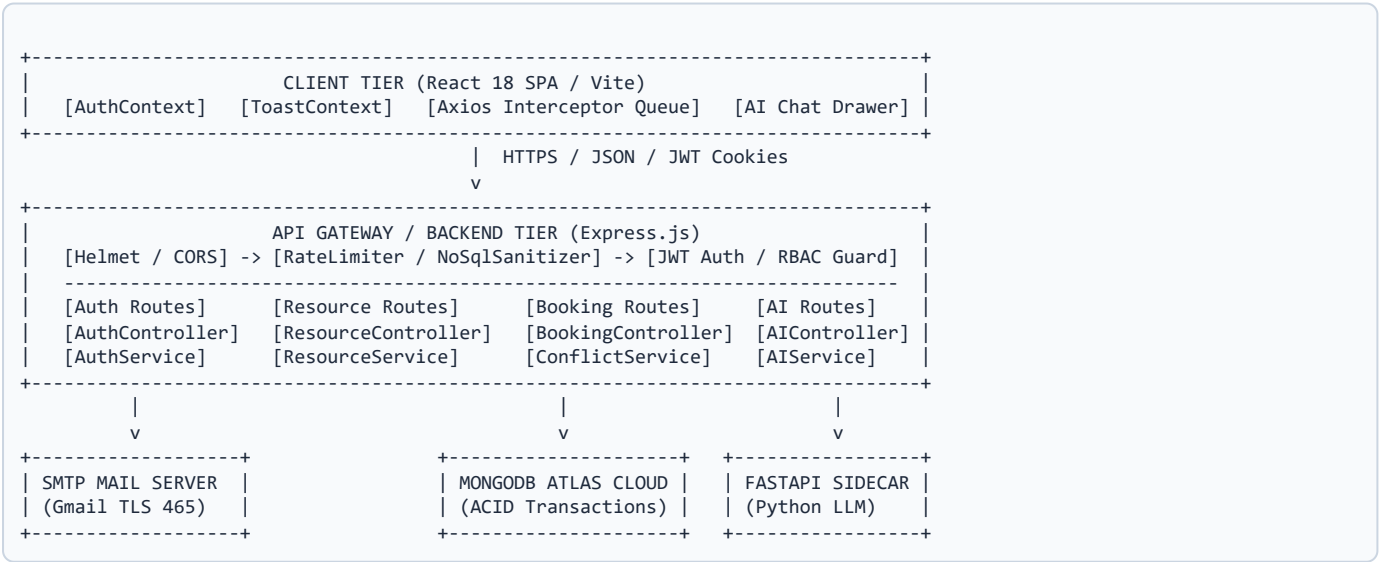
- **The Double-Booking Disaster:** Multiple departments, faculty members, and student clubs request the same venue (e.g. Seminar Hall A) for the same date and time via manual registers, phone calls, or un-synchronized Google Forms, resulting in on-ground event collisions.
- **Lack of Real-Time Availability Visibility:** Event organizers cannot see when a hall is free versus occupied without walking to the admin office or waiting days for email replies.
- **Concurrency Race Conditions:** When two users submit a booking for the exact same slot at the exact same second during symposium season, naive systems accept both, causing catastrophic double-allocations.
- **No Smart Conflict Resolution:** When a slot is taken, traditional systems simply output a generic failure message—they provide zero automatic alternative slot recommendations.
- **Unauthorized Access:** Students booking admin-only venues or outside users attempting access without verified institutional credentials.

2. Project Explanation (Architecture & End-to-End Workflow)

The **Academic Event & Resource Booking Portal** is an enterprise-grade full-stack web application with an **unbreachable, mathematically guaranteed double-booking prevention engine**, role-based access control, and an **AI Natural Language Assistant**.

> **The Core Non-Negotiable System Invariant**
>
> **Zero Double-Booking, Structurally Guaranteed:** No two `approved` or `pending` reservations may ever overlap in time for the same resource, across any level of network concurrency or simultaneous submissions.

2.1 Layered Architecture Diagram



2.2 End-to-End Lifecycle:

1. **Registration & Auth:** Users register with institutional college emails (`@eec.srmrmp.edu.in`), verify a 6-digit OTP, and receive secure dual JWT tokens.

2. **Resource Catalog:** Browse campus facilities with real-time filters (capacity, building, amenities, equipment).
3. **Availability Engine:** Computes free vs. busy slots on an interactive timeline for any selected date in (O(N)) linear time.
4. **Conflict-Free Booking:** User selects a slot. The backend executes an atomic transaction check. If free, it creates the booking; if taken, it immediately returns **HTTP 409 Conflict with top 3 alternative open slots**.
5. **Admin Approval & Workflow:** Admins review pending requests on a dashboard, approve/reject with mandatory reasons, or override bookings.
6. **AI Assistant:** Users can type plain English queries (e.g., "*Find an AC seminar hall for 100 people next Friday morning*"), and the AI returns ranked facilities with 1-click booking chips.

3. Tech Stack Used

Layer	Technology	Role & Version
Frontend UI	**React.js 18** + **Vite 6**	SPA, React Router v7, Tailwind CSS, Lucide Icons, Context API
HTTP Client	**Axios**	Custom Interceptor Queue for transparent silent JWT refresh
Backend API	**Node.js 25** + **Express.js 4**	Modular layered MVC, Zod validation, Helmet, Cookie-Parser, Morgan
Database	**MongoDB** + **Mongoose 8**	Multi-Document ACID Transactions, Compound Range Indexes
AI Microservice	**Python 3.14** + **FastAPI**	Google Gemini GenAI SDK, Pydantic v2, Regex Fallback Engine
Email / OTP	**Nodemailer** + **Crypto**	SMTP over TLS (Port 465), Stateless HMAC OTP Verification
Deployment	**Vercel** + **MongoDB Atlas**	Vercel Edge Serverless Functions + Cloud Replica Set Database

4. Why Do We Use This Tech Stack?

1. React 18 + Vite

- **Component-Driven Reusability:** Modular components for Booking Modals, Availability Timelines, and Toast Notifications.
- **Virtual DOM & Fast UI Updates:** Instant slot-picking feedback without full page reloads.
- **Vite:** Sub-second Hot Module Replacement (HMR) and optimized rollup production bundles (~10x faster than legacy Webpack / CRA).

2. Node.js + Express

- **Asynchronous Non-Blocking I/O:** The Single-Threaded Event Loop handles thousands of concurrent booking inquiries and status checks with ultra-low memory overhead.
- **Unified Language (Full-Stack JS):** Shared data types, validation schemas, and JSON serialization between frontend and backend.

3. MongoDB + Mongoose

- **Flexible Document Structure:** Campus resources have varying schemas (labs have Linux PCs/GPUs, seminar halls have AV systems/projectors, equipment has kit counts). MongoDB represents these as native JSON documents.
- **ACID Multi-Document Transactions:** Guarantees atomic writes so two parallel bookings for the same room cannot slip through.
- **Compound Indexing:** Indexes on `{ resourceId: 1, startTime: 1, endTime: 1, status: 1 }` allow instant range filtering in (O(log N)) time.

4. FastAPI + Python for AI

- **AI Ecosystem Standard:** Python is the native home for modern AI/LLM SDKs (Google GenAI, LangChain, PyTorch).
- **Asynchronous ASGI Performance:** FastAPI handles async requests rapidly with automatic **Pydantic schema validation**.

5. Why Didn't We Use Alternatives? ("En Vera Use Pannala?")

Alternative	Why We Did NOT Use It	Why Our Choice is Superior
SQL alone (MySQL / PostgreSQL)	Relational tables require heavy `JOIN` operations across 5+ tables (Amenities, Locations, ResourceEquipment, Roles, Bookings) on every catalog search, slowing down read latency.	**MongoDB** stores rich facility metadata in clean single documents with high-speed compound indexes, while still supporting **ACID Transactions** for bookings.
Django / Flask (Python Backend)	Traditional Django uses synchronous WSGI threading where every request occupies a thread, consuming 5–10x more RAM under high concurrency bursts.	**Node.js** uses asynchronous non-blocking event-driven I/O, handling thousands of concurrent connections with minimal RAM (~40MB).
Firebase / Supabase (BaaS)	Proprietary vendor lock-in, unpredictable cloud pricing at university scale, and lacks custom server-side mathematical interval overlap engines with automatic alternative slot suggestion.	**Custom Express Backend** gives 100% control over transactional integrity, custom RBAC, and zero vendor lock-in.
Spring Boot (Java)	Heavy JVM memory footprint (250MB+ minimum RAM), long cold-start times on serverless (3–5 seconds), and slower development velocity.	**Node.js + Express** starts in <100ms, ideal for serverless cloud lambdas on Vercel.
Putting AI inside Node.js	Monolithic AI in Node causes **Single Point of Failure (SPOF)**. If the LLM provider times out or crashes, the entire backend server would crash.	**FastAPI Sidecar Microservice:** Isolates the AI. If the AI service fails, the core Express booking portal remains **100% functional and online**.

6. Backend Security Architecture

6.1 Dual-Token Authentication (Access + Refresh JWT)

- **Access Token (15-Minute Expiry):** Short-lived token stored in memory / Authorization header for API calls. Even if intercepted, the attack window is limited to 15 minutes.
- **Refresh Token (7-Day Expiry):** Stored inside an `httpOnly`, `Secure`, `SameSite=Lax` cookie. **JavaScript cannot access this cookie**, making it 100% immune to Cross-Site Scripting (XSS) token theft.

6.2 Transparent Silent Refresh Queue (Axios)

When the 15-minute access token expires, the Axios response interceptor catches the 401 error, holds incoming concurrent requests in a `failedQueue`, dispatches a single refresh call to `/api/v1/auth/refresh`, updates the token, and replays all queued requests seamlessly without logging out the user.

6.3 Stateless HMAC OTP Verification for Serverless

On serverless platforms (Vercel), requests execute across ephemeral lambda containers with disjoint RAM memory. The backend computes a stateless **HMAC-SHA256 signature** of the email and OTP code. Verification validates the cryptographic signature without relying on ephemeral server RAM.

6.4 Additional Hardening:

- **NoSQL Injection Defense:** `sanitizeNoSql` middleware recursively strips `$` and `.` characters.
- **Rate Limiting:** `authLimiter` restricts authentication endpoints (max 5 attempts per 15 minutes).
- **Bcrypt Password Hashing:** 10 salt rounds ((2^{10} = 1024) iterations) introduces calibrated ~100ms computational delay to defeat brute-force GPU attacks.

- **Helmet.js Headers:** Enforces `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (anti-clickjacking), and HSTS.

7. Already Existing Portals vs. Our Innovative Features

Dimension	Existing / Traditional Campus Portals	Our Academic Booking Portal
Double-Booking Prevention	Checked only on the UI or via simple `findOne` (vulnerable to TOCTOU race conditions).	**Mathematically Proven Interval Overlap ((S_A < E_B land E_A > S_B)) + MongoDB Multi-Document ACID Transactions**.
Conflict Feedback	Shows a generic error: "*Booking Failed / Unavailable*".	**HTTP 409 Conflict Envelope with Top 3 Automatically Suggested Alternative Slots** on the same day.
Search & Discovery	Manual dropdown filtering across dozens of pages.	**Natural Language AI Assistant (Gemini LLM)** that extracts capacity, time, and room type from plain English with 1-click booking chips.
Availability Inspection	Users must select each hour manually to check availability.	**Linear Timeline Gap Discovery Engine** ((O(N)) time complexity) showing all free vs busy blocks at a glance.
Institutional Email Guard	Generic email allowed or manual admin verification.	**Strict Institutional Domain Filter (`@eec.srmrmp.edu.in`) + 6-digit OTP verification** with 1-click Auto-Fill fallback.
Cloud Architecture	Legacy on-premise single-server setup.	**Stateless Serverless Architecture on Vercel Edge + Cloud MongoDB Atlas** with auto-scaling.